LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4: INHERITANCE

Dibuat Oleh :

Nuevalen Refitra Alswando 1030724300008

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM SURABAYA

2026

1. **Access Modifier - Public dan Private**

Manusia.java

```java
package modul04.AccessModifier;

public class Manusia {
    private String nama;
    private int umur;

    public void setNama(String a) { nama = a; }

    public String getNama() {
        return nama;
    }

    public void setUmur(int a) { umur = a; }

    public int getUmur() { return umur; }
}
```

DemoManusia.java

```java
package modul04.AccessModifier;

public class DemoManusia {
    public static void main(String[] args) {
        Manusia objMns1 = new Manusia();

        objMns1.setNama("Markonah");
        objMns1.setUmur(76);

        System.out.println("Nama: " + objMns1.getNama());
        System.out.println("Umur: " + objMns1.getUmur());

        Manusia objMns2 = new Manusia();
        objMns2.setNama("Budi");
        objMns2.setUmur(25);

        System.out.println("\nNama: " + objMns2.getNama());
        System.out.println("Umur: " + objMns2.getUmur());

        // Coba akses langsung (akan error)
        // objMns1.nama = "Test"; // ERROR: nama has private access
        // objMns1.umur = 30;     // ERROR: umur has private access
    }
}
```

Output

```
Run    DemoManusia
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\li
Nama: Markonah
Umur: 76

Nama: Budi
Umur: 25

Process finished with exit code 0
aktikum > src > modul04 > AccessModifier > DemoManusia > main
```

Implementasi enkapsulasi dilakukan untuk melindungi data internal objek dengan cara menggunakan modifier private pada atribut seperti nama dan umur, sehingga tidak dapat diakses langsung dari luar class atau dikenal sebagai data hiding. Sebagai antarmuka untuk mengakses atribut private tersebut, method getter dan setter dideklarasikan dengan modifier public. Objek dibuat menggunakan keyword new, di mana akses terhadap atribut private hanya diperbolehkan melalui method public (getter dan setter) yang telah disediakan. Apabila mencoba mengakses atribut private secara langsung dari luar class, maka akan terjadi error compile yang menandakan bahwa prinsip enkapsulasi telah berjalan dengan benar.

2. Constructor dengan Berbagai Parameter

Manusia.java

Manusia.java DemoManusia.java

```java
package modul04.ConstructorParameter;

public class Manusia {  8 usages  new *
    private String nama;  4 usages
    private int umur;  3 usages

    public Manusia() {  1 usage  new *
        System.out.println("Constructor default dipanggil");
    }

    public Manusia(String a) {  1 usage  new *
        nama = a;
        System.out.println("Constructor dengan 1 parameter dipanggil");
    }

    public Manusia(String a, int b) {  1 usage  new *
        nama = a;
        umur = b;
        System.out.println("Constructor dengan 2 parameter dipanggil");
    }

    public void setNama(String a) {  new *
        nama = a;
    }

    public String getNama() {  new *
        return nama;
    }

    public void setUmur(int a) {  1 usage  new *
        umur = a;
    }

    public int getUmur() {  1 usage  new *
        return umur;
    }
}
```

DemoManusia.java

```java
package modul04.ConstructorParameter;

public class DemoManusia {
    public static void main(String[] args) {
        Manusia arrMns[] = new Manusia[3];

        Manusia objMns1 = new Manusia();
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);

        Manusia objMns2 = new Manusia("Mat Conan");

        Manusia objMns3 = new Manusia("Bajuri", 13);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        System.out.println("=== DATA MANUSIA ===");
        for(int i = 0; i < 3; i++) {
            System.out.println("Nama: " + arrMns[i].getNama());
            System.out.println("Umur: " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=49793" -Dfile.encoding
Constructor default dipanggil
Constructor dengan 1 parameter dipanggil
Constructor dengan 2 parameter dipanggil
=== DATA MANUSIA ===
Nama: Markonah
Umur: 76

Nama: Mat Conan
Umur: 0

Nama: Bajuri
Umur: 13
```

Pada implementasi kode tersebut, class didefinisikan dengan memiliki 3 constructor yang memiliki parameter berbeda, suatu konsep yang dikenal sebagai constructor overloading. Constructor pertama bersifat default tanpa parameter, constructor kedua dirancang untuk menginisialisasi atribut nama, sedangkan constructor ketiga digunakan untuk menginisialisasi nama dan umur sekaligus. Saat objek dibuat, program secara otomatis memilih constructor mana yang akan dijalankan sesuai dengan parameter masukan yang diberikan. Fleksibilitas ini menunjukkan bahwa constructor overloading memungkinkan pembuatan objek dengan berbagai cara inisialisasi tergantung kebutuhan. Selain itu, penggunaan Array of Object juga diterapkan yang memungkinkan penyimpanan beberapa objek dalam satu variabel array untuk mengelola banyak instance sekaligus.

3. **Protected Access Modifier**

Parent.java

```java
package modul04.ProtectedAccessModifier.test;

public class Parent {
    public int publicInt;
    private int privateInt;
    int defaultInt;
    protected int protectedInt;

    public void setPrivateInt(int privateInt) {
        this.privateInt = privateInt;
    }
}
```

Child.java

```java
package modul04.ProtectedAccessModifier.other;


import modul04.ProtectedAccessModifier.test.Parent;

public class Child extends Parent {
    public void methodChild() {
        publicInt = 10;          // ✓ OK: public dapat diakses dari mana saja
        setPrivateInt(20);       // ✓ OK: akses private via setter method
        protectedInt = 30;       // ✓ OK: protected dapat diakses subclass (walaupun beda package)
        // defaultInt = 40;      // × ERROR: default hanya bisa diakses dalam package yang sama
    }
}
```

Access modifier dalam Java menentukan tingkat aksesibilitas terhadap atribut dan method. Variabel **publicInt** yang menggunakan modifier public dapat diakses dari mana saja tanpa batasan. Variabel **privateInt** dengan modifier private memiliki akses paling terbatas, yaitu hanya dapat diakses dalam class Parent itu sendiri, sehingga untuk mengaksesnya dari class lain diperlukan setter method sebagai perantara. Variabel **defaultInt** yang tidak menggunakan modifier apapun (default) hanya dapat diakses oleh class-class yang berada dalam package yang sama. Sementara itu, variabel **protectedInt** dengan modifier protected memiliki karakteristik khusus yaitu dapat diakses oleh semua class dalam package yang sama dan juga dapat diakses oleh subclass meskipun berada di package yang berbeda. Dalam konteks inheritance, Child class yang berada di package berbeda tetap dapat mengakses protectedInt milik Parent class, namun tidak dapat mengakses defaultInt

karena defaultInt terbatas pada package yang sama. Untuk privateInt, meskipun Child class mewarisi Parent class, akses terhadap privateInt tetap dibatasi dan hanya dapat dilakukan melalui setter method yang bersifat public.

4. Package - Mengorganisasi Class

HargaToken.java

```java
package modul04.MengorganisasiClas.hargatoken;

public class HargaToken {
    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

HargaPulsa.java

```java
package modul04.MengorganisasiClas.hargapulsa;

public class HargaPulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

Main.java

```java
package modul04.MengorganisasiClas.main;

import modul04.MengorganisasiClas.hargatoken.HargaToken;
import modul04.MengorganisasiClas.hargapulsa.HargaPulsa;

public class Main {
    public static void main(String[] args) {
        HargaToken objectToken = new HargaToken();
        objectToken.info();

        System.out.println();

        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=49825" -Dfile.enc
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000

Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000

Process finished with exit code 0
```

Dalam implementasi program, class dideklarasikan dalam package hargatoken untuk mengelompokkan class yang berkaitan dengan harga token listrik, sementara class lainnya dideklarasikan dalam package hargapulsa untuk class yang menangani harga pulsa. Penggunaan package sangat bermanfaat karena membantu mengorganisasi class berdasarkan kategori tertentu, sehingga struktur program menjadi lebih terstruktur dan mudah dikelola, terutama ketika jumlah class yang dibuat sangat banyak. Di dalam class hargapulsa, terdapat method info() yang berfungsi untuk menampilkan daftar harga pulsa yang tersedia. Untuk mengakses class dari package yang berbeda, seperti ketika class Main yang berada di package main ingin menggunakan class dari package hargatoken dan hargapulsa, diperlukan penggunaan keyword import di awal program. Class dari package berbeda dapat digunakan dan diakses selama access modifier yang digunakan pada class tersebut memungkinkan, seperti menggunakan access modifier public yang dapat diakses dari package manapun.

5. Non-Access Modifier – Final

FinalExample.java

```java
package modul04.NonAccessModifier;

public class FinalExample {
    public static void main(String[] args) {
        final int a = 5;
        final int b;
        b = 8;

        System.out.println("Nilai a: " + a);
        System.out.println("Nilai b: " + b);

        // a = 15;  // ERROR: final variable tidak bisa diubah
        // b = 21;  // ERROR: final variable sudah diinisialisasi
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=49829" -Dfile.enc
Nilai a: 5
Nilai b: 8

Process finished with exit code 0
```

Variabel yang menggunakan modifier final harus diinisialisasi sebanyak satu kali saja dan setelah itu nilainya tidak dapat diubah kembali. Proses inisialisasi ini dapat dilakukan langsung saat deklarasi variabel atau dilakukan kemudian pada bagian kode lainnya, namun tetap hanya diperbolehkan satu kali inisialisasi. Karena memiliki sifat nilai yang tetap dan tidak dapat berubah sepanjang eksekusi program, variabel final berperilaku sebagai konstanta dalam program Java.

6. Final pada Method dan Class

```java
package modul04.NonAccessModifier;

public class FinalParent {
    public void publicMethod() { System.out.println("Public method"); }

    public final void finalMethod() { System.out.println("Final method - cannot be overridden"); }
}
```

```java
package modul04.NonAccessModifier;

public class FinalChild extends FinalParent {
    @Override
    public void publicMethod() { System.out.println("Public method overridden"); // OK }

    // @Override
    // public void finalMethod() {  // ERROR: final method tidak bisa di-override
    //     System.out.println("This will cause error");
    // }
}
```

FinalClass.java

```java
package modul04.NonAccessModifier;

public final class FinalClass {
    public void someMethod() { System.out.println("This class cannot be extended"); }
}

// class AnotherClass extends FinalClass {  // ERROR: FinalClass tidak bisa di-extend
// }
```

Penggunaan modifier final memiliki batasan spesifik terhadap method dan class dalam hierarki inheritance. Pada method, jika dideklarasikan sebagai final, maka method tersebut tidak dapat di-override oleh subclass, berbeda dengan public method biasa yang dapat di-override secara normal menggunakan annotation @Override. Apabila memaksa untuk melakukan overriding pada finalMethod, maka akan menyebabkan error compile yang menandakan pelanggaran aturan tersebut. Sementara itu, jika modifier final diterapkan pada class, class tersebut tidak dapat di-extend oleh class lain. Hal ini berguna untuk mendefinisikan class yang tidak ingin dimodifikasi atau diturunkan perilakunya melalui mekanisme inheritance, sehingga memastikan integritas implementasi class tersebut tetap terjaga.

7. Static Modifier

```java
package modul04.StaticModifier;

public class StaticExample {
    public static int count = 0;

    public static void incrementCount() { count++; }

    public int instanceValue = 0;

    public static void main(String[] args) {
        System.out.println("Count awal: " + StaticExample.count);

        StaticExample.incrementCount();
        StaticExample.incrementCount();

        System.out.println("Count setelah increment: " + StaticExample.count);

        StaticExample obj1 = new StaticExample();
        StaticExample obj2 = new StaticExample();

        obj1.instanceValue = 10;
        obj2.instanceValue = 20;

        System.out.println("obj1.instanceValue: " + obj1.instanceValue);
        System.out.println("obj2.instanceValue: " + obj2.instanceValue);
        System.out.println("Static count (shared): " + StaticExample.count);
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=64620" -Dfile.enc
Count awal: 0
Count setelah increment: 2
obj1.instanceValue: 10
obj2.instanceValue: 20
Static count (shared): 2

Process finished with exit code 0
```

Modifier static dalam Java digunakan untuk mendefinisikan variabel dan method yang menjadi milik kelas itu sendiri, bukan milik instance objek. Karena variabel static dimiliki oleh class, bukan instance, maka semua objek yang dibuat dari kelas tersebut akan berbagi static variable yang sama sehingga nilainya bersifat global untuk seluruh instance. Selain itu, static method dapat dipanggil tanpa membuat objek terlebih dahulu karena method ini tidak memerlukan instant object untuk menjalankannya. Namun, terdapat batasan penting dimana static method tidak dapat mengakses instance variable secara langsung karena static method tidak dapat mengacu pada variabel yang bukan static atau milik instance.

8. Implementasi Lengkap Enkapsulasi

Mahasiswa.java

```java
package modul04.Enkapsulasi;
public class Mahasiswa {
    private String nim;
    private String nama;
    private String jurusan;
    private double ipk;
    public Mahasiswa() {
        System.out.println("Mahasiswa object created");
    }
    public Mahasiswa(String nim, String nama, String jurusan) {
        this.nim = nim;
        this.nama = nama;
        this.jurusan = jurusan;
        this.ipk = 0.0;
    }
    public String getNim() {
        return nim;
    }
    public void setNim(String nim) {
        this.nim = nim;
    }
    public String getNama() {
        return nama;
    }
    public void setNama(String nama) {
        this.nama = nama;
    }
    public String getJurusan() {
        return jurusan;
    }
    public void setJurusan(String jurusan) {
        this.jurusan = jurusan;
    }
    public double getIpk() {
        return ipk;
    }
    public void setIpk(double ipk) {
        if(ipk >= 0.0 && ipk <= 4.0) {
            this.ipk = ipk;
        } else {
            System.out.println("IPK tidak valid! Harus antara 0.0 - 4.0");
```

```java
public class Mahasiswa {
    public void setNim(String nim) {
        this.nim = nim;
    }
    public String getNama() {
        return nama;
    }
    public void setNama(String nama) {
        this.nama = nama;
    }
    public String getJurusan() {
        return jurusan;
    }
    public void setJurusan(String jurusan) {
        this.jurusan = jurusan;
    }
    public double getIpk() {
        return ipk;
    }
    public void setIpk(double ipk) {
        if(ipk >= 0.0 && ipk <= 4.0) {
            this.ipk = ipk;
        } else {
            System.out.println("IPK tidak valid! Harus antara 0.0 - 4.0");
        }
    }
    public void tampilkanInfo() {
        System.out.println("=== INFO MAHASISWA ===");
        System.out.println("NIM: " + nim);
        System.out.println("Nama: " + nama);
        System.out.println("Jurusan: " + jurusan);
        System.out.println("IPK: " + ipk);
        System.out.println();
    }
}
```

MainMahasiswa.java

```java
package modul04.Enkapsulasi;

public class MainMahasiswa {
    public static void main(String[] args) {
        Mahasiswa[] mhs = new Mahasiswa[3];

        mhs[0] = new Mahasiswa("2024001", "Ahmad Rizki", "Teknik Informatika");
        mhs[0].setIpk(3.75);

        mhs[1] = new Mahasiswa("2024002", "Siti Nurhaliza", "Sistem Informasi");
        mhs[1].setIpk(3.90);

        mhs[2] = new Mahasiswa();
        mhs[2].setNim("2024003");
        mhs[2].setNama("Budi Santoso");
        mhs[2].setJurusan("Ilmu Komputer");
        mhs[2].setIpk(3.50);

        System.out.println("=== DAFTAR MAHASISWA ===\n");
        for(int i = 0; i < mhs.length; i++) {
            mhs[i].tampilkanInfo();
        }

        System.out.println("=== TEST VALIDASI IPK ===");
        mhs[0].setIpk(5.0);

        // Test akses langsung (akan error)
        // mhs[0].nim = "12345";  // ERROR: nim has private access
        // mhs[0].ipk = 4.5;      // ERROR: ipk has private access
    }
```

Ouput

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=64615" -Dfile.enc
Mahasiswa object created
=== DAFTAR MAHASISWA ===

=== INFO MAHASISWA ===
NIM: 2024001
Nama: Ahmad Rizki
Jurusan: Teknik Informatika
IPK: 3.75

=== INFO MAHASISWA ===
NIM: 2024002
Nama: Siti Nurhaliza
Jurusan: Sistem Informasi
IPK: 3.9

=== INFO MAHASISWA ===
NIM: 2024003
Nama: Budi Santoso
Jurusan: Ilmu Komputer
IPK: 3.5

=== TEST VALIDASI IPK ===
IPK tidak valid! Harus antara 0.0 - 4.0

Process finished with exit code 0
```

Pada implementasi kode tersebut, seluruh atribut dideklarasikan menggunakan access modifier private untuk mendukung konsep enkapsulasi dan menyembunyikan data internal dari akses langsung. Khusus untuk atribut IPK, method setter dilengkapi dengan validasi logika yang memastikan nilai yang dimasukkan berada dalam rentang valid antara 0.0 hingga 4.0, sehingga integritas data tetap terjaga. Fleksibilitas dalam pembuatan objek dicapai

melalui mekanisme constructor overloading, yang mendemonstrasikan penggunaan baik constructor default maupun constructor dengan parameter sesuai kebutuhan inisialisasi. Selain itu, program juga mengimplementasikan Array of Object untuk efisiensi dalam menyimpan dan mengelola data multiple mahasiswa dalam satu variabel. Adanya validasi IPK melalui method setter ini secara nyata menunjukkan keuntungan utama enkapsulasi, yaitu data internal objek menjadi terlindungi dari input yang tidak valid atau akses langsung yang tidak diinginkan.